Universidad Mayor de San Andrés

Facultad de Ciencias Puras y Naturales

Carrera de Informática

Sistema multiplataforma de inventario para una tienda con base de datos en la nube

Inventario Cloud

Proyecto de la materia

Materia: Computación en la Nube

Estudiantes: Jorge Alberto Ichuta Tuco

Alex Ronaldo Huayllucu Huayllucu

José Fernando Ignacio Mamani

David Jhon Mamani Vallejo

Cesar Raymundo Claros Choquemisa

Docente: Ph. D. José María Tapia Baltazar

La Paz – Bolivia, octubre de 2026

1 Introducción

El control del inventario es una de las tareas más importantes de cualquier tienda, porque de él dependen las compras a los proveedores, la atención a los clientes y, en el fondo, saber cuánto dinero está guardado en los estantes. A pesar de eso, en muchas tiendas de barrio, minimarkets y negocios pequeños de La Paz y El Alto el inventario todavía se lleva en un cuaderno, en una hoja de Excel o en un archivo que vive en una sola computadora.

Este tipo de registro funciona mientras el negocio es pequeño, pero con el tiempo empieza a fallar: las cantidades anotadas no coinciden con lo que hay en el almacén, se repiten registros, se pierden hojas y nadie sabe con certeza qué productos están por agotarse. Además, si el dueño está fuera de la tienda o si dos empleados venden al mismo tiempo, no hay forma sencilla de compartir la información actualizada.

Durante el semestre, en la materia de Computación en la Nube, estudiamos cómo los servicios en la nube permiten acceder a recursos de cómputo y almacenamiento desde cualquier lugar, sin necesidad de comprar ni mantener servidores propios. Tomando esa idea como base, en el presente proyecto proponemos **Inventario Cloud**, un sistema multiplataforma de inventario para una tienda que funciona como aplicación Android (instalada mediante un archivo APK) y como aplicación web desde el navegador de una computadora.

Ambas versiones se desarrollan con **Flutter** y se conectan a una sola base de datos **PostgreSQL** alojada en **Supabase**, utilizando únicamente su plan gratuito. Así, una venta registrada desde el celular del empleado se refleja en la computadora del administrador, y todo el proyecto puede desarrollarse y demostrarse con un presupuesto de **Bs 0**.

El documento está organizado de la siguiente manera: en la sección 2 se presenta el marco teórico que sustenta la propuesta; en la sección 3 se describe la problemática y se identifica el problema central; en la sección 4 se plantean los objetivos del proyecto; y finalmente, en la sección 5 se detallan los servicios en la nube que utiliza el sistema y los servicios que este ofrece a sus usuarios.

2 Marco teórico

2.1 Sistemas de control de inventario

Un **inventario** es el conjunto de bienes que una empresa mantiene para venderlos o utilizarlos en sus operaciones [4]. Controlarlo significa registrar de forma ordenada todas las **entradas** (compras o reposiciones de mercadería) y todas las **salidas** (ventas,

mermas o ajustes), de modo que en cualquier momento se conozca la **existencia** o **stock** de cada producto. Para nuestro sistema son importantes los siguientes conceptos:

- **Stock actual:** cantidad disponible de un producto en un momento dado.
- **Stock mínimo:** nivel por debajo del cual conviene reponer el producto para no quedarse sin existencias. Cuando el stock actual es igual o menor que el mínimo, el sistema genera una **alerta de bajo stock**.
- **Movimiento de inventario:** registro de cada cambio en las existencias, con fecha, usuario, producto, cantidad, tipo (entrada, venta o ajuste), stock anterior y stock resultante. El conjunto de movimientos forma el **historial** o *kardex* del producto.
- **Ajuste:** corrección de las existencias cuando el conteo físico no coincide con el registro. En lugar de borrar movimientos, se registra un movimiento nuevo, con lo que se conserva la trazabilidad.

2.2 Computación en la nube

Según el Instituto Nacional de Estándares y Tecnología de los Estados Unidos (NIST), la computación en la nube es un modelo que permite el acceso ubicuo, conveniente y bajo demanda, a través de la red, a un conjunto compartido de recursos informáticos configurables (redes, servidores, almacenamiento, aplicaciones y servicios), que pueden aprovisionarse y liberarse rápidamente con un mínimo esfuerzo de gestión [6]. La misma definición establece cinco **características esenciales**:

1. **Autoservicio bajo demanda:** el usuario obtiene recursos sin intervención humana del proveedor.
2. **Amplio acceso a la red:** los servicios se consumen desde celulares, computadoras u otros dispositivos a través de Internet.
3. **Agrupación de recursos:** el proveedor comparte su infraestructura entre muchos clientes.
4. **Elasticidad rápida:** los recursos aumentan o disminuyen según la necesidad.
5. **Servicio medido:** el consumo se mide y, según el plan, se cobra o se limita.

Armbrust et al. [1] destacan que la principal ventaja de la nube para organizaciones pequeñas es que convierte un gasto inicial grande (comprar servidores) en un gasto variable según el uso, que en proyectos pequeños puede llegar a ser nulo gracias a los planes gratuitos.

Modelos de servicio

Los servicios en la nube se clasifican según cuánto de la infraestructura administra el proveedor y cuánto el cliente. El cuadro 1 resume los modelos más conocidos.

Modelo	Qué ofrece el proveedor	Ejemplos
IaaS (Infraestructura como servicio)	Máquinas virtuales, almacenamiento y redes; el cliente instala y administra el sistema operativo y las aplicaciones.	Amazon EC2, Google Compute Engine
PaaS (Plataforma como servicio)	Un entorno listo para desplegar aplicaciones sin administrar servidores.	Netlify, Cloudflare Pages
SaaS (Software como servicio)	Aplicaciones completas listas para usar desde el navegador.	GitHub, Gmail, Google Drive
BaaS (Backend como servicio)	Base de datos, autenticación, API y almacenamiento ya integrados; el desarrollador solo construye la interfaz.	Supabase, Firebase

Cuadro 1: Modelos de servicio en la nube.

Modelos de despliegue

La nube puede ser **pública** (servicios de un proveedor ofrecidos a cualquier cliente por Internet), **privada** (infraestructura exclusiva de una organización), **comunitaria** (compartida por organizaciones con intereses comunes) o **híbrida** (combinación de las anteriores) [6]. Nuestro sistema utiliza una **nube pública** con un modelo **BaaS**, lo que nos evita comprar y mantener un servidor propio.

2.3 Supabase como backend en la nube

Supabase es una plataforma de código abierto del tipo BaaS construida sobre PostgreSQL [9]. Cada proyecto de Supabase incluye una base de datos PostgreSQL dedicada, un servicio de autenticación de usuarios, una API REST que se genera automáticamente a partir de las tablas, la posibilidad de llamar funciones de la base de datos (RPC), almacenamiento de archivos, notificaciones en tiempo real y funciones *serverless*. Su plan gratuito permite desarrollar prototipos sin costo, dentro de los límites que se detallan en la sección 5.

2.4 Bases de datos relacionales y PostgreSQL

Una **base de datos relacional** organiza la información en tablas relacionadas mediante claves primarias y foráneas, lo que garantiza la **integridad referencial**: por ejemplo, no puede existir una venta de un producto que no está registrado [8]. PostgreSQL es un sistema gestor de bases de datos relacional, libre y de código abierto [7], que ofrece varias características que aprovechamos en el proyecto:

- **Transacciones ACID** (atomicidad, consistencia, aislamiento y durabilidad): una venta con varios productos se guarda completa o no se guarda.
- **Restricciones** (`CHECK`, `UNIQUE`, `NOT NULL`) que impiden, por ejemplo, existencias negativas o códigos de producto repetidos.
- **Funciones almacenadas** que ejecutan varias operaciones en una sola transacción, como registrar una venta, descontar el stock y crear el movimiento.
- **Bloqueo de filas** (`SELECT ... FOR UPDATE`) para evitar que dos ventas simultáneas vendan la misma última unidad.
- **Row Level Security (RLS):** políticas que deciden, fila por fila, qué datos puede leer o modificar cada usuario según su rol.

2.5 API REST, HTTPS y JSON

Una **API** (interfaz de programación de aplicaciones) es el intermediario que permite que la aplicación se comunique con la base de datos sin conectarse a ella directamente. El estilo arquitectónico **REST**, propuesto por Fielding [2], organiza la comunicación en recursos a los que se accede con los métodos de HTTP (`GET`, `POST`, `PATCH`, `DELETE`). En nuestro caso los datos viajan en formato **JSON** y a través de **HTTPS**, que cifra la comunicación entre el dispositivo y el servidor.

Usar una API tiene una ventaja importante de seguridad: las credenciales de la base de datos nunca se guardan en los celulares ni en las computadoras, y se puede controlar qué operaciones realiza cada usuario.

2.6 Autenticación y autorización

La **autenticación** comprueba la identidad del usuario (por ejemplo, con correo y contraseña), mientras que la **autorización** determina qué puede hacer una vez dentro del sistema. En Supabase, al iniciar sesión el usuario recibe un **token JWT** [5] firmado que acompaña a cada petición; la base de datos lee ese token para aplicar las políticas RLS. Las contraseñas no se guardan en texto plano, sino como un **hash** administrado por el propio servicio de autenticación.

El sistema aplica un **control de acceso basado en roles** con dos perfiles: **administrador**, que gestiona todo el inventario y los usuarios, y **empleado**, que consulta productos y registra ventas.

2.7 Desarrollo multiplataforma con Flutter

Flutter es un *framework* gratuito y de código abierto creado por Google que permite construir aplicaciones para Android, iOS, web y escritorio a partir de un **único**

código fuente escrito en el lenguaje Dart [3]. En este proyecto lo usamos para generar dos versiones del mismo sistema:

- Un archivo **APK** que se instala directamente en celulares Android, sin necesidad de publicarlo en Google Play.
- Una **aplicación web** que se abre desde Chrome, Edge o Firefox en cualquier computadora.

Ambas versiones comparten la lógica del negocio y adaptan la interfaz al tamaño de la pantalla, lo que se conoce como diseño adaptable o *responsive*.

2.8 Disponibilidad y respaldo de la información

Centralizar los datos en la nube mejora la **disponibilidad**, porque la información puede consultarse desde cualquier lugar con Internet, y la **sincronización**, porque todos los dispositivos trabajan sobre la misma base de datos. Sin embargo, estar en la nube no reemplaza a los **respaldos**: es necesario exportar periódicamente el esquema y los datos para poder recuperarlos ante un error humano o la pérdida del proyecto, más aún cuando el plan gratuito no incluye copias automáticas [10].

3 Problemática – Identificación del problema

3.1 Descripción de la situación actual

En muchas tiendas pequeñas y medianas el inventario se administra con cuadernos, hojas de cálculo o archivos guardados en una sola computadora. Cada empleado anota las ventas a su manera, las entradas de mercadería se registran tarde (o directamente no se registran) y la única copia de la información depende de un solo equipo.

Como consecuencia, el responsable de la tienda no sabe en tiempo real cuántas unidades tiene de cada producto. Algunos productos se agotan sin que nadie lo note a tiempo, mientras que otros se compran de más y se quedan en el almacén. Revisar el historial de entradas y ventas, o saber qué proveedor trajo cada producto, se vuelve una tarea lenta y poco confiable.

3.2 Identificación del problema

A partir de la situación descrita identificamos el siguiente **problema central**:

Problema central

La tienda no cuenta con un control centralizado de su inventario, por lo que la información sobre existencias, entradas y ventas es poco confiable y no está disponible al mismo tiempo para todos los dispositivos y usuarios.

Para analizarlo utilizamos un **árbol de problemas** (figura 1), donde en la parte inferior se ubican las causas, al centro el problema y en la parte superior los efectos que produce.

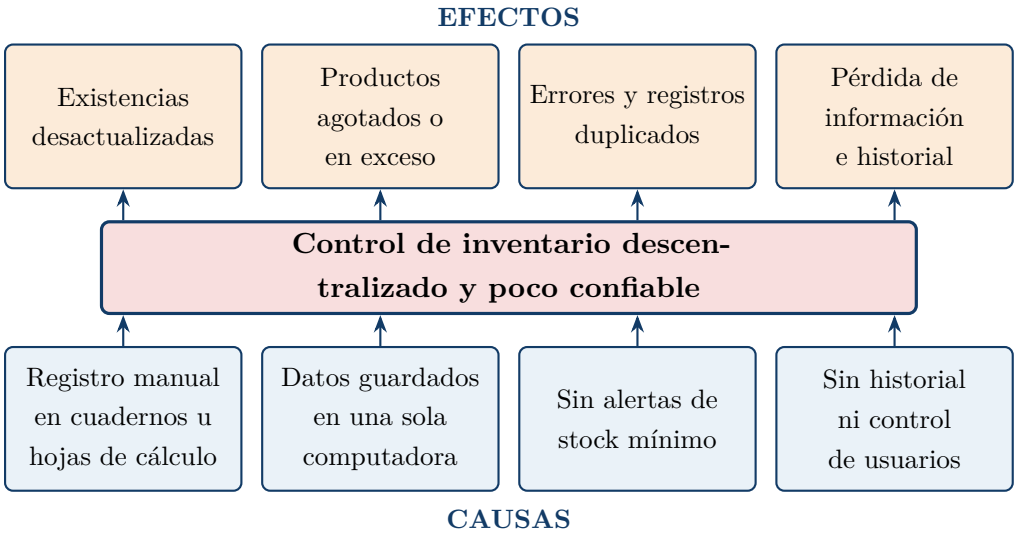


Figura 1: Árbol de problemas del control de inventario.

3.3 Análisis de causas y consecuencias

En el cuadro 2 relacionamos cada causa con su consecuencia y con la forma en que el sistema propuesto la atiende.

Causa	Consecuencia	Respuesta del sistema
Registro manual de entradas y ventas	Errores de captura, duplicados y stock incorrecto	Formularios con validación y actualización automática del stock
Información en una sola computadora	No se puede consultar desde otros dispositivos; riesgo de pérdida	Base de datos única en la nube, accesible desde Android y PC
Ausencia de alertas	Productos agotados sin aviso	Alerta cuando el stock es igual o menor que el mínimo
Sin historial de movimientos	No se sabe cuándo ni por qué cambió el stock	Registro automático de cada entrada, venta y ajuste
Sin control de usuarios	Cualquiera puede modificar los datos	Inicio de sesión y permisos por rol
Sin copias de seguridad	Pérdida definitiva ante una falla	Respaldos manuales periódicos de la base de datos

Cuadro 2: Causas, consecuencias y respuesta del sistema propuesto.

3.4 Formulación del problema

A partir del análisis anterior, formulamos el problema de investigación con la siguiente pregunta:

> *¿Cómo centralizar y mantener actualizada la información del inventario de una tienda, de forma que pueda consultarse y modificarse de manera segura y sincronizada desde celulares Android y computadoras, sin incurrir en costos de infraestructura?*

4 Objetivos

4.1 Objetivo general

Desarrollar un sistema multiplataforma de inventario para una tienda, accesible desde celulares Android y computadoras, conectado mediante una API a una base de datos en la nube, que permita registrar, organizar y consultar productos, categorías, proveedores, existencias, entradas y ventas, con el propósito de mejorar el control del inventario, reducir errores y facilitar el acceso sincronizado a la información desde diferentes dispositivos.

4.2 Objetivos específicos

1. Diseñar una base de datos relacional que almacene la información de productos, categorías, proveedores, existencias y movimientos de inventario.
2. Diseñar una interfaz adaptable que pueda utilizarse desde teléfonos celulares y computadoras.
3. Implementar las funciones necesarias para registrar, consultar, actualizar y desactivar la información del sistema.
4. Registrar las entradas de mercadería y las ventas realizadas para mantener actualizadas las existencias de cada producto.
5. Generar alertas cuando la cantidad disponible de un producto sea igual o menor que el nivel mínimo establecido.
6. Mantener un historial de movimientos que permita conocer cuándo y por qué aumentaron o disminuyeron las existencias.
7. Alojar la base de datos en un servicio de nube para permitir el acceso remoto y centralizado a la información.
8. Sincronizar los cambios realizados desde celulares y computadoras mediante una única base de datos en la nube.
9. Aplicar medidas básicas de seguridad, como autenticación de usuarios, control de acceso por rol y protección de las credenciales de conexión.
10. Configurar mecanismos de respaldo y recuperación para disminuir el riesgo de pérdida de información.
11. Realizar pruebas que comprueben el funcionamiento, la disponibilidad y la integridad de los datos almacenados.

5 Lista de servicios y su descripción

Inventario Cloud se apoya en servicios de nube gratuitos para funcionar y, a su vez, ofrece un conjunto de servicios a los usuarios de la tienda. La figura 2 muestra cómo se relacionan: los dos clientes (celular y computadora) se comunican por HTTPS con Supabase, que concentra la autenticación, la API y la base de datos.

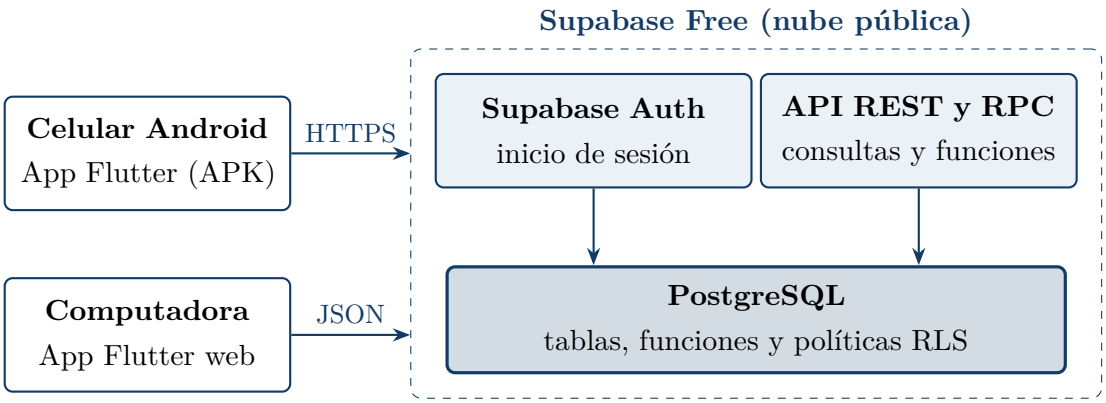


Figura 2: Arquitectura de servicios de Inventario Cloud.

5.1 Servicios en la nube utilizados

En el cuadro 3 describimos cada servicio de nube que utiliza el sistema, el modelo de servicio al que pertenece y la función que cumple dentro de la solución.

Todo el proyecto se diseña con un presupuesto de **Bs 0**. A octubre de 2026, el plan gratuito de Supabase ofrece los recursos del cuadro 4 [10], que son suficientes para el prototipo y la demostración.

5.2 Modelo de la base de datos

La base de datos del sistema está en PostgreSQL, dentro de Supabase. Antes de crear las tablas hicimos el modelo entidad-relación, que se muestra en la figura 3. Cada recuadro es una tabla y debajo del nombre están sus campos. **PK** es la clave primaria de la tabla y **FK** es una clave foránea, o sea, un campo que apunta a otra tabla. En las líneas, el **1** y la **N** indican cuántos registros se relacionan de cada lado.

Servicio	Modelo	Descripción y uso en el sistema
Supabase Database (PostgreSQL)	BaaS	Base de datos relacional en la nube. Guarda usuarios, productos, categorías, proveedores, entradas, ventas y movimientos, y contiene las funciones que registran cada operación en una sola transacción.
Supabase Auth	BaaS	Autenticación con correo y contraseña. Guarda las contraseñas como hash y entrega a cada usuario un token de sesión con el que la base identifica su rol.
API REST de Supabase	BaaS	API generada automáticamente a partir de las tablas. La aplicación la usa para consultar y modificar catálogos mediante HTTPS y JSON.
Funciones RPC	BaaS	Permiten llamar desde la aplicación a `registrar_entrada`, `registrar_venta` y `registrar_ajuste`, que validan el stock y actualizan las existencias.
Row Level Security	BaaS	Políticas que limitan qué filas puede leer o modificar cada rol. Protegen los datos incluso si alguien llama a la API sin usar la aplicación.
Supabase Storage (opcional)	BaaS	Almacenamiento de archivos para las fotografías de los productos.
Supabase Realtime (opcional)	BaaS	Avisa a los dispositivos conectados cuando cambian los datos, para actualizar el stock en pantalla sin refrescar.
Supabase Edge Functions (opcional)	FaaS	Funciones *serverless* para tareas con permisos administrativos, como crear cuentas desde la aplicación.
Netlify o Cloudflare Pages	PaaS	Alojamiento gratuito de la versión web (resultado de `flutter build web`).
GitHub	SaaS	Repositorio del código fuente y de los scripts SQL, con control de versiones y trabajo por ramas.

Cuadro 3: Servicios en la nube utilizados por el sistema.

Recurso	Límite gratuito	Consideración para el proyecto
Base de datos	500 MB por proyecto	Suficiente para cientos de productos y movimientos
Almacenamiento de archivos	1 GB	Solo si se agregan fotos de productos
Transferencia de datos	5 GB	Las consultas en JSON son pequeñas
Proyectos activos	2	Uno principal y uno para pruebas de recuperación
Respaldos automáticos	No incluidos	Haremos respaldos manuales
Inactividad	Pausa tras 1 semana sin uso	Ingresar al sistema antes de cada presentación

Cuadro 4: Límites del plan gratuito de Supabase.

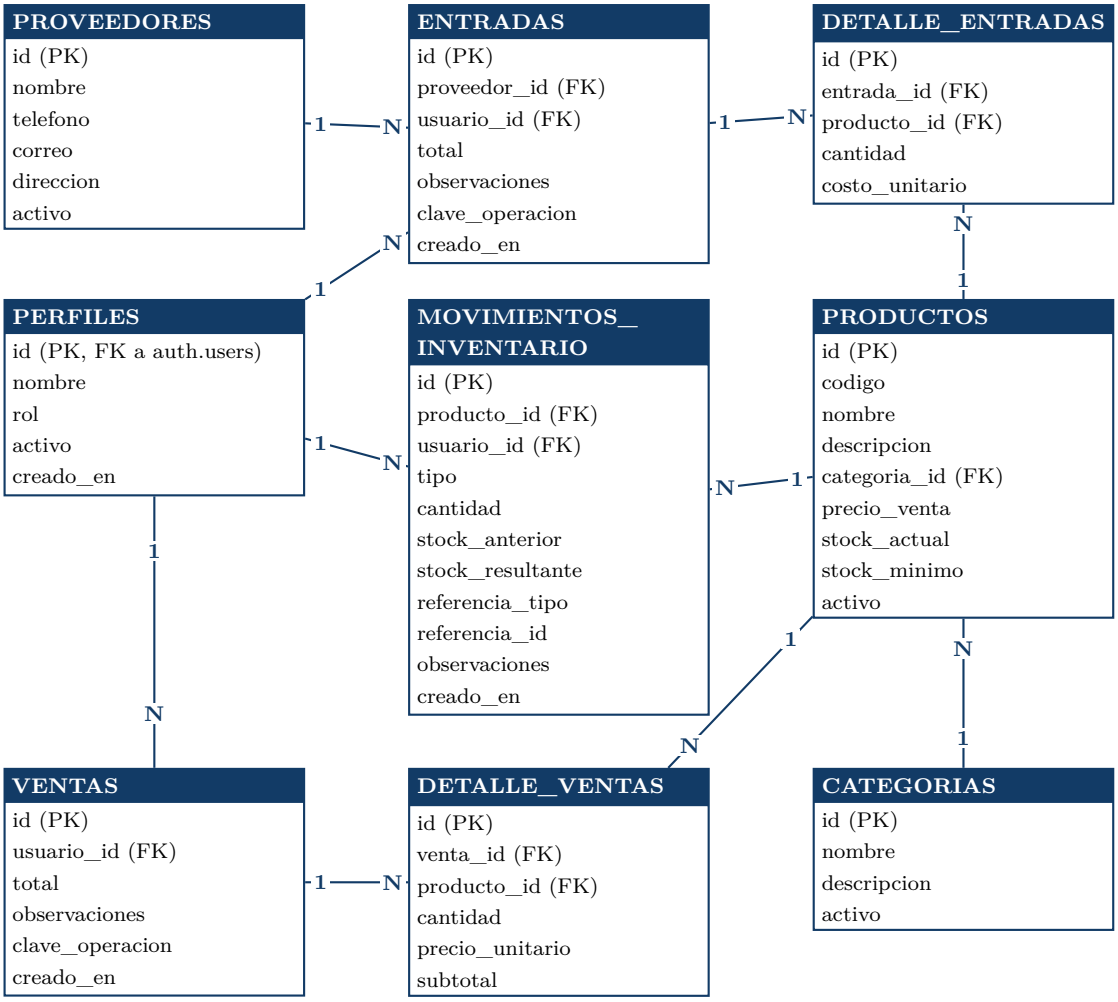


Figura 3: Modelo entidad-relación de la base de datos.

Las relaciones del modelo se leen así:

- Una categoría tiene muchos productos, y cada producto pertenece a una sola categoría.
- Un proveedor puede tener muchas entradas de mercadería.
- Cada entrada y cada venta tienen su detalle, donde se guarda qué productos entraron o se vendieron, en qué cantidad y a qué precio.
- Un usuario puede registrar muchas entradas, ventas y movimientos.
- Cada vez que cambia el stock de un producto se guarda un registro en movimientos de inventario. Así sabemos quién hizo el cambio, cuándo lo hizo y cuánto stock había antes y después.

La tabla de perfiles está unida a la tabla de usuarios de Supabase (`auth.users`), que es donde se guardan los correos y las contraseñas. En nuestras tablas no guardamos ninguna contraseña. Además, el stock de los productos no se puede cambiar a mano: solo cambia cuando se registra una entrada, una venta o un ajuste.

5.3 Servicios que ofrece el sistema

Desde el punto de vista del usuario de la tienda, Inventario Cloud ofrece los siguientes servicios:

Autenticación de usuarios

Inicio y cierre de sesión con correo y contraseña. Al ingresar, el sistema lee el rol del usuario y muestra solo las funciones que le corresponden.

Gestión de usuarios y roles

El administrador consulta los usuarios, asigna el rol de administrador o empleado, y activa o desactiva cuentas.

Gestión de categorías

Registro, consulta, edición y desactivación de las categorías que agrupan a los productos.

Gestión de proveedores

Registro y consulta de los proveedores con sus datos de contacto (teléfono, correo y dirección).

Gestión de productos

Registro de productos con código único, nombre, categoría, precio de venta, stock actual y stock mínimo. Los productos que ya tienen movimientos se desactivan en lugar de eliminarse.

Búsqueda de productos

Búsqueda por nombre o código y, como función opcional, lectura del código de barras con la cámara del celular.

Registro de entradas de mercadería

El administrador registra la compra a un proveedor con uno o varios productos, cantidades y costos; el stock aumenta automáticamente.

Registro de ventas

Administradores y empleados registran ventas con uno o varios productos. El sistema calcula el total, descuenta el stock y rechaza la venta si no hay existencias suficientes.

Control de existencias y ajustes

El stock de cada producto se mantiene siempre actualizado y nunca puede quedar en negativo. El administrador puede corregir diferencias con un ajuste que queda registrado.

Alertas de bajo stock

El panel principal y una pantalla propia muestran los productos cuyo stock es igual o menor que el mínimo.

Historial de movimientos

Consulta de todas las entradas, ventas y ajustes con fecha, usuario, producto, cantidad y stock resultante. El empleado solo ve sus propios movimientos.

Sincronización multiplataforma

Todos los dispositivos trabajan sobre la misma base de datos, por lo que una venta hecha en el celular se ve en la computadora al actualizar la pantalla.

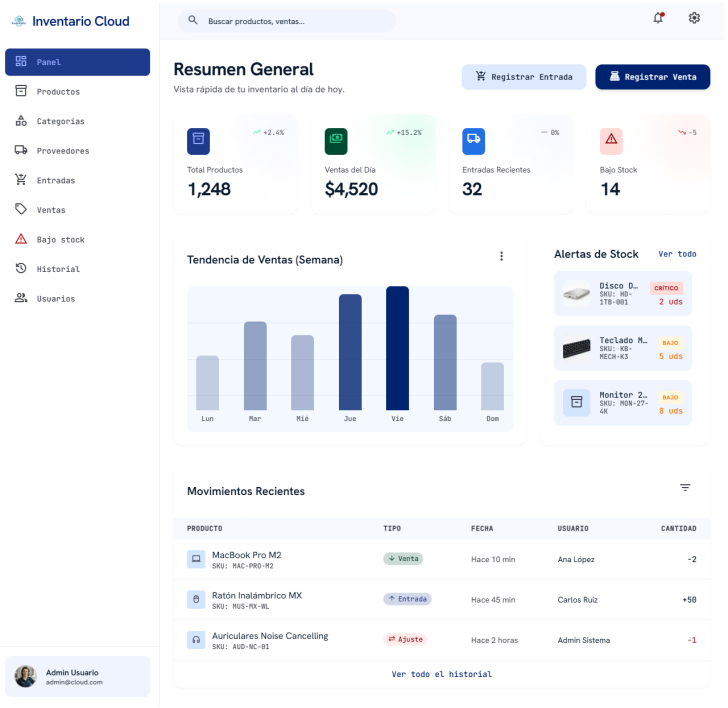
Respaldo de la información

Exportación periódica del esquema y los datos para recuperarlos en caso de error.

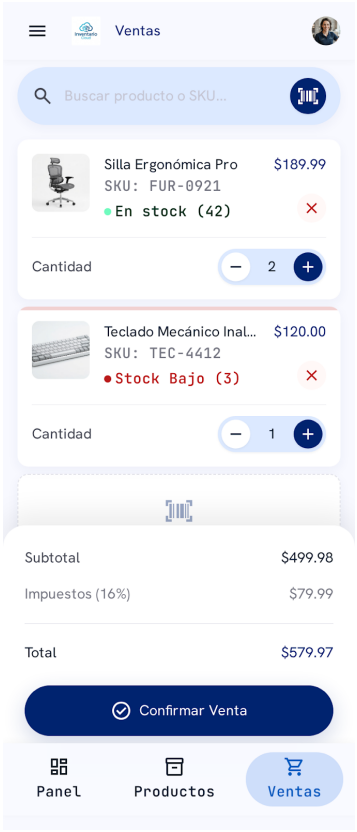
Reportes (opcional)

Gráficas de los productos más vendidos y exportación de reportes a CSV o PDF.

La figura 4 muestra los prototipos de interfaz que elaboramos en la etapa de diseño: el panel principal en la versión de escritorio y el registro de una venta en la versión móvil. Los valores que aparecen son datos de ejemplo.



(a) Panel principal, versión web



(b) Registro de venta, versión Android

Figura 4: Prototipos de interfaz de Inventario Cloud.

Referencias

[1] Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I. y Zaharia, M. (2010). A view of cloud computing. *Communications of the ACM*, 53(4), 50–58.

[2] Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures*. Tesis doctoral, University of California, Irvine.

[3] Google. (s.f.). *Flutter documentation*. https://docs.flutter.dev

[4] Heizer, J. y Render, B. (2014). *Principios de administración de operaciones* (9.ª ed.). Pearson Educación.

[5] Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

[6] Mell, P. y Grance, T. (2011). *The NIST definition of cloud computing* (Special Publication 800-145). National Institute of Standards and Technology.

[7] PostgreSQL Global Development Group. (s.f.). *PostgreSQL documentation: Row security policies*. https://www.postgresql.org/docs/current/ddl-rowsecurity.html

[8] Silberschatz, A., Korth, H. F. y Sudarshan, S. (2020). *Database system concepts* (7.ª ed.). McGraw-Hill.

[9] Supabase. (s.f.). *Supabase documentation*. https://supabase.com/docs

[10] Supabase. (2026). *Pricing*. Consultado el 21 de octubre de 2026. https://supabase.com/pricing